LPS: longest proper prefix of S[0..i] that is a suffix of S[0..i].

LPS[0] = 0 because a length 1 string has no proper prefix.

Prefix "A" = suffix "A" ⇒ length 1

Prefix "AB" ≠ suffix "BA"

the only prefix is "A" which is not a suffix

LPS = [0 , 0]

S =	A	B	A	B	C	A	B	A	B
	0	1	2	3	4	5	6	7	8

LPS = [0 , 0 , 1]

S =	A	B	A	B	C	A	B	A	B
	0	1	2	3	4	5	6	7	8

Algorithm: two-Pointers

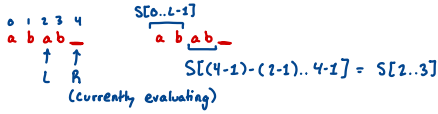
- R represents the end of the suffix we are currently evaluating.
- L represents the length of the matching prefix to the suffix ending at R-1.

from prior iteration.. not the to-be-evaluated R index.

→ because of 0-based indexing, it also represents the index of the next expected char for this prefix.

Invariants:

- S[0..L-1] = S[(R-1)-(L-1).. R-1]
- LPS[R-1] = L (length)



if S[L] = S[R] then this char matches the expected next prefix-suffix char.

↳ update invariant: LPS[R] = L+1

↳ prepare next iteration's evaluation: L++, R++

else: we cannot extend the current prefix.

if L = 0: ++R
else: L = LPS[L-1]

S = A B A C A B A B

L R (currently evaluating)

↳ since S[L] ≠ S[R], we cant extend the prefix.

↳ However, it's possible that a smaller suffix could still exist ending at R.
Since we only know that S[L] ≠ S[R], we can try to see if the longest prefix-suffix ending before this character (L-1), is a valid prefix-suffix ending at R.

↳ L = LPS[L-1]

↳ A B A C A B A B ⇒ A B A C A B A B ✓

L, old R for L=1, R

the next iteration will find this match, otherwise if S[L] ≠ S[R] again, we would perform the same operation until L=0.

* the base case is when L=0, and S[L] still doesnt match S[R], we simply increment R.